

El lenguaje ensamblador del MIPS R2000

Ing. Walter Lozano
Ing. Alejandro Rodríguez Costello

Junio 2019

1. Uso de registros

Ahora abordaremos la carga y almacenamiento de datos entre memoria y los registros del procesador. Dado que la arquitectura del R2000 es RISC, utiliza un subconjunto concreto de instrucciones que permiten las acciones de carga y almacenamiento de los datos entre los registros del procesador y la memoria. Generalmente, las instrucciones de carga de un dato de memoria a registro comienzan con la letra “l” (de load en inglés) y las de almacenamiento de registro en memoria con “s” (de store en inglés), seguidos por la letra inicial correspondiente al tamaño del dato que se va a mover, b para byte, h para media palabra (half word) y w para palabra (word).

1.1. Carga de datos inmediatos

En el siguiente código la directiva `.text [dir]` sirve para indicar el comienzo de la zona de memoria dedicada a las instrucciones. Por defecto esta zona comienza en la dirección `0x00400000` si no indicamos ningún argumento en `.text`. A continuación se encuentran las instrucciones que introducimos en nuestro programa. La primera de ellas se referencia con la etiqueta `main` y le indica al simulador dónde este es el principio del programa que debe ejecutar. Por defecto hace referencia a la dirección `0x00400020`. A partir de esta dirección, el simulador cargará el código de nuestro programa en el segmento de *memoria de instrucciones*.

```
        .text                #zona de instrucciones
main:    lui    $s0, 0x8690
```

Para cargar un valor en un registro podemos hacerlo introduciendo una constante en la propia instrucción. Dicho **modo de direccionamiento** se le llama **inmediato**.

La instrucción `lui` es la única instrucción de carga inmediata real. Tiene dos *operandos*, el primero es el registro destino, en este caso `s0`, y el segundo una constante de 16 bits (half word). Esta carga dicha constante en la parte alta del registro especificado. La parte baja del registro correspondiente a los bits de menor peso se pone a 0.

Carga el programa anterior y localiza la instrucción en la memoria de instrucciones y reconoce la dirección donde se encuentra, el tamaño que ocupa y la instrucción en código máquina equivalente, analizando cada campo de esta y

cuál es el tipo de formato. Luego ejecútala comprobando el efecto en el estado de `s0` perteneciente al fichero de registros del procesador.

El ensamblador del MIPS ofrece la posibilidad de cargar una constante de 32 bits en un registro utilizando la pseudoinstrucción `li` (*load immediate*). Esta pseudo instrucción se descompone en otras instrucciones propias del procesador. Utilizando el siguiente código comprueba qué conjunto de instrucciones reales implementan esta pseudoinstrucción y como afecta el programa al registro involucrado.

```
        .text
main:    li $s0, 0x12345678
```

Finalmente a veces es necesario cargar en un registro una constante que refiere a una dirección de memoria de 32 bits. Esto podría hacerse con la pseudoinstrucción anterior pero implica que el programador debe especificar dicha dirección en forma explícita (su valor numérico). En assembly es comun especificar las direcciones mediante etiquetas y por lo tanto existe una pseudoinstrucción específica para cargar direcciones de memoria en un registro utilizando como operando una etiqueta. Dicha pseudo instrucción se llama `la` (*load address*). Utilizando el siguiente código comprueba qué conjunto de instrucciones reales implementan esta pseudoinstrucción y como afecta el programa al registro involucrado. Presta atención al valor de la etiqueta `number`.

```
        .data
number:  .word 0x12345678
        .text
main:    la $s0, number
```

1.2. Carga de datos de memoria a registros

En la sección anterior la carga de información en registros proviene siempre de un valor constante almacenado de alguna manera en una o más instrucciones dependiendo del tamaño de la misma.

En cambio para cargar información desde la memoria a los registros se utiliza un registro auxiliar llamado registro **base** que contiene la dirección de memoria donde se encuentra el dato a cargar y opcionalmente una constante que se suma a la dirección de memoria que contiene el registro base. A este modo se le llama modo de direccionamiento **base**.

Vamos a tomar como ejemplo como cargar un entero en un registro. La instrucción `lw` del próximo código carga en el registro `s0` la palabra contenida en una posición de memoria cuya dirección se especifica en el registro `t0`.

```
        .data
number:  .word 0x12345678
        .text
main:    la $t0, number
        lw $s0, 0($t0)
```

Localiza la instrucción `lw` en la memoria de instrucciones y observa cómo ha transformado dicha instrucción en código máquina el simulador analizando cada campo de esta y su tipo de formato. Luego ejecuta el código, inspecciona como altera cada registro y que valor queda en `s0`.

Existen formas alternativas que equivalen al código anterior. Estas proporcionan mayor legibilidad al programa pero complican su depuración (*debugging*) ya que en general se descomponen en instrucciones diferentes utilizando algún registro auxiliar. En el siguiente código la posición de memoria se especifica como un desplazamiento dado por la etiqueta **number** respecto del registro 0. La dirección del dato se calcula sumando el contenido del registro 0, que siempre vale cero, y dicha etiqueta.

```
.data
number: .word 0x12345678
.text
main:   lw $s0, number($0)
```

Otra alternativa es no especificar un registro base como se observa en el siguiente código. Ambas formas son muy útiles cuando la dirección de memoria del dato a buscar está bien definida con una etiqueta y no es dinámica, es decir no cambiaremos el valor de la base durante la ejecución del programa.

```
.data
number: .word 0x12345678
.text
main:   lw $s0, number
```

Reflexiona sobre cómo transforma el compilador el código especificado en cada variante y como especifica realmente la dirección del dato a partir de la dirección de la etiqueta **number**. ¿Por qué crees que el simulador traduce de esta forma la instrucción de carga?

También puede cargarse desde memoria half words con la instrucción **lh** que permite cargar 16 bits desde memoria a un registro en los 16 bits de menor peso del mismo o bytes con la instrucción **lb** que permite cargar 8 bits desde memoria a un registro en los 8 bits de menor peso.

Intenta modificar el primer programa de esta sección para que guarde en el registro **s0** los dos bytes de mayor peso de **number** y repite los análisis anteriores y luego haz lo mismo con el byte de mayor peso.

1.2.1. Datos no alineados

Si modifica el código anterior para que en lugar de transferir los dos bytes de mayor peso de **number**, se transfiera los dos bytes que están contenidos en la dirección referenciada por **number+1** como se muestra en el código a continuación, ¿qué ocurre al ejecutarse y por qué?

```
.data
number: .word 0x12345678
.text
main:   lh $s0, number+1
```

Se observa en la consola de mensajes del simulador que retorna un error **Runtime exception at** indicando la dirección del código donde se produce el error, en este caso 0x400004, y luego el motivo como se observa a continuación, impidiendo que el programa no termina su ejecución normalmente.

```
: fetch address not aligned on halfword boundary 0x10010001
```

Tanto la instrucción `lw` como `lh` esperan que los datos estén alineados en posiciones de memoria múltiplo de 4 o 2 respectivamente. En cambio el siguiente código no retorna ningún error. Reflexione el porqué.

```
        .data
number:  .word 0x12345678
        .text
main:    lb $s0, number+1
```

1.3. Almacenado de datos en memoria

El almacenamiento de información en la memoria a partir de datos en el interior de los registros, es la operación contraria a la carga de información desde la memoria. Cada operación de carga `lw`, `lh` y `lb` tiene su homólogo para guardar la información en memoria: `sw`, `sh` y `sb` que utilizan el mismo modo de direccionamiento y permiten las mismas variaciones sintácticas.

En el siguiente código `sw` almacena la palabra contenida en un registro en una dirección de memoria. Esta dirección se obtiene de la misma manera que se obtiene con `lw`, sumando el contenido de un registro base más un desplazamiento especificado en la instrucción.

```
        .data
number:  .word 0x10203040
place:   .space 4
other:   .word 0xffffffff
        .text
main:    lw $s0, number
        sw $s0, place
        sw $s0, other
```

Carga el código en el simulador y localiza la primera instrucción de este tipo en la memoria de instrucciones y analiza cómo ha transformado dicha instrucción el simulador. Luego ejecútalo y comprueba el efecto de la ejecución del programa en la memoria de datos.

Ahora te proponemos que investigues por tu cuenta la acción de `sh` y `sb`. Para ello te proponemos que construyas los siguientes ejemplos realizando los mismos análisis indicados en oportunidades anteriores:

- Almacena el half word de menor peso de un registro en una dirección de memoria
- Almacena el byte de menor peso de un registro en una dirección de memoria.
- Comprueba que ocurre si almacenas distintos datos: word, half word y byte en direcciones no alineadas y reflexiona porqué algunos representan errores o no.

1.4. Problemas propuestos de la sección

1. Diseña un programa ensamblador que defina un vector de enteros que contiene los valores 10, 20, 25, 500, 3 en la memoria de datos a partir de

la dirección 0x10000000 y cargue todos sus elementos en los registros `$s0` al `$s4` y luego lo copie a partir de la dirección 0x10010000.

2. Diseña un programa ensamblador que, dada la palabra 0x10203040 almacenada en una posición de memoria, la reorganice en otra posición de memoria, invirtiendo el orden de sus bytes y luego la reorganize en otra posición a nivel half word.
3. Diseña un programa ensamblador que inicialice cuatro bytes a partir de la posición 0x10010002 a los siguientes valores 0x10, 0x20, 0x30, 0x40, y reserve espacio para una palabra a partir de la dirección 0x1001010. El programa transferirá los cuatro bytes contenidos a partir de la posición 0x10010002 a la dirección 0x1001010.

2. Operaciones

Una de las funciones principales de un procesador es hacer operaciones aritméticas, lógicas y de desplazamiento. Aquí se pondrán en juego los conocimientos sobre representación de la información.

2.1. Operaciones aritméticas

Las instrucciones aritméticas están constituidas por las operaciones usuales de suma, resta, multiplicación y división. Debemos recordar que en una computadora la representación de la información es finita y en particular en MIPS R2000 está limitado por registros de 32 bits. Por lo tanto el rango de representación de los números enteros con signo en complemento a 2 va desde -2.147.483.648 a 2.147.483.647 (0x80000000 a 0x7fffffff).

Las instrucciones aritméticas por defecto utilizan el modo de direccionamiento por **registros**, este modo implica que solo se pueden utilizar registros como operandos. Algunas instrucciones pueden utilizar el modo inmediato, con lo cual uno de sus operandos puede ser una constante especificada en la propia instrucción.

Los resultados siempre se almacena en un registro destino especificado por la instrucción o en forma implícita en registros especializados como en el caso de la multiplicación y división.

Las instrucciones aritméticas por defecto son signadas, utilizando operandos representados en complemento a dos y si el resultado de la operación no es representable en 32 bits provocan una excepción de desbordamiento. Las variantes no signadas no tienen en cuenta el desbordamiento y utilizan la letra adicional “u” para indicar *unsigned*.

El formato más general de la suma es `add rd rs rt` donde `rs` y `rt` representan los dos operadores en complemento a dos `rd` el registro destino donde se almacena el resultado. En el siguiente ejemplo de código se cargan dos números enteros en los registros `s0` y `s1`, `add` suma el contenido de `s0` y `s1` colocando el resultado en `s2` y lo guarda en la posición de memoria `num3`¹.

¹De aquí en adelante se entiende que debe cargar todos los ejemplos de código en el simulador, ejecutarlos y hacer los análisis habituales

```

        .data
num1:    .word 0x2
num2:    .word 0x4
num3:    .space 4
        .text
main:    lw $s0, num1
        lw $s1, num2
        add $s2, $s0, $s1
        sw $s2, num3

```

Es importante destacar que los registros fuente y destino no tienen porque ser distintos como se muestra en el ejemplo a continuación donde la suma de `s0` más `s0` y se almacena en `s0`.

```

        .data
num1:    .word 0x2
        .text
main:    lw $s0, num1
        add $s0, $s0, $s0

```

Las instrucciones de suma pueden utilizar el modo de direccionamiento inmediato con o sin signo como se indicó anteriormente. En el siguiente ejemplo se utiliza la operación suma sin detección de desbordamiento `addiu` para sumar una constante 1 al máximo número positivo representable. Ahora reflexiona que ocurre si reemplazas `addiu` por `addi` y por qué.

```

        .data
num1:    .word 2147483647
        .text
main:    lw $s0, num1
        addiu $s1, $s0, 1

```

El siguiente ejemplo se utiliza la instrucción de resta `subu` para decrementar en 1 dos veces el máximo número negativo representable en complemento a 2 y se guarda el resultado en `num2`. Reflexiona sobre la consigna y medita si el resultado es correcto. Reemplaza algún `subu` por `sub` y verifica que ocurre.

```

        .data
num1:    .word 0x80000000
num2:    .word 1
        .text
main:    lw $s0, num1
        lw $s1, num2
        subu $s0, $s0, $s1
        subu $s0, $s0, $s1
        sw $s0, num1

```

Es importante entender las consecuencias sobre el significado de la información en la memoria. Si en la consigna anterior cambiamos el significado de `num1` diciendo que en el mismo se almacena el número entero sin signo `0x80000000` ¿Que opinas ahora sobre la correctitud del resultado?

Aunque `sub` no soporta el modo de direccionamiento inmediato, la misma existe como pseudoinstrucción. Modifica el programa anterior para utilizar `subiu`. Reflexiona sobre las ventajas o desventajas de la misma.

2.1.1. Multiplicación y división

Estas dos operaciones en MIPS R2000 merecen un poco más de detalle por dos motivos:

1. En general la multiplicación binaria de dos números de 32 bits requiere un espacio de 64 bits para almacenar el resultado, mientras que la división requiere 32 bits para el resultado y 32 bits adicionales para el resto.
2. El ISA del MIPS R2000 no admite en el formato de sus instrucciones especificar más de 1 registro destino.

Por lo tanto estas operaciones guardan sus resultados en registros especializados llamados `hi` y `lo` que no están mapeados directamente en el banco de registros y por lo tanto no son directamente accesibles con las instrucciones de carga y descarga de registros a memoria estudiadas.

Para entender como trabaja veamos el siguiente código que realiza la multiplicación de dos números, almacenando su resultado a continuación de los dos multiplicandos. La instrucción `mult` multiplica `s0` con `s1` y guarda los 32 bits más significativos en `hi` y los siguientes bits en `lo`. Luego se utilizan las instrucciones `mfhi` para mover `hi` a `s0` y guardarlo en los primeros 32 bits reservados por `.space` y finalmente `mflo` se mueve `lo` a `s1` para guardarlo en los bytes restantes. Cuando hagas el análisis del código observa que como 16 es una potencia de 2 su multiplicación produce un efecto equivalente al desplazamiento en 4 bits que es muy facil de observar en el segmento de memoria.

```
        .data
num1:   .word 0x7FFFFFFF
num2:   .word 16
        .space 8
        .text
main:   lw $s0, num1
        lw $s1, num2
        mult $s0, $s1
        mfhi $s0
        mflo $s1
        sw $s0, num2+4
        sw $s1, num2+8
```

Modifica el código anterior para dividir `num1` por `num2` sabiendo que `div` almacena el resto en `hi` y el resultado en `lo` y guardando primero el resultado y luego el resto.

Cabe destacar que existen pseudoinstrucciones tanto para la multiplicación, como para la división que permiten especificar un registro fuente. Usando la ayuda del simulador explora las mismas y verifica cuales son sus limitaciones.

2.2. Operaciones lógicas

A esta altura el lector debe tener conocimiento de las operaciones básicas del álgebra booleana que posiblemente haya visto representadas a nivel bit con dos variables de entrada. Cuando hablamos de operaciones lógicas en un procesador como MIPS R2000 nos referimos a esas mismas operaciones llevadas a

cabo ordenadamente bit a bit entre los bits que componen dos registros fuentes almacenando el resultado en el correspondiente bit del registro destino. Para la suma lógica disponemos de **or** y **ori**, para suma lógica exclusiva **xor** y **xori** y para el producto lógico **and** y **andi**, que por default usan el modo de direccionamiento por registro y las terminadas en “i” usan el modo de direccionamiento inmediato, cuya sintaxis es similar a las operaciones aritméticas vistas. Aunque el procesador no dispone de **not** como instrucción nativa, si dispone de **nor** que se define como la negación de la suma y por lo tanto dado que $0+x = x$ entonces se puede implementar **not** como pseudoinstrucción.

El siguiente ejemplo pone los 16 bits más significativos y el bit 0 de la palabra contenida en **number** a 0 y almacena el resultado en la siguiente palabra.

```

        .data
number:  .word 0x3ff41
        .space 4
        .text
main:    lw $s0, number
        andi $s1, $s0, 0xfffe
        # 0xfffe == 0...0111111111111110
        sw $s1, number+4

```

La instrucción **andi**, realiza el producto lógico, bit a bit, entre el dato contenido en **s0** y la constante **0xfffe**. Aquellos bits que en **0xfffe** valen 1 no modifican el resultado con respecto al contenido original del registro, es decir, los 16 bits de menor peso excepto el bit 0. Por otro lado, todos aquellos bits que están a 0 en la constante, también están a 0 en el resultado, es decir, los 16 bits de mayor peso y el bit 0. Recordar que para la operación lógica *and* valen las expresiones $0.x = 0$ y $1.x = x$.

Aclaración: Aunque la constante que se almacena en la instrucción **andi** es de 16 bits, a la hora de realizar la operación el procesador convierte el dato inmediato en una constante de 32 bits, poniendo los 16 bits de mayor peso a 0 y fijando por tanto los 16 bits de mayor peso del resultado también a 0.

Modifica el código para obtener, en la siguiente palabra, que los 16 bits más significativos de numero permanezcan tal cual, y que los 16 bits menos significativos queden a uno, excepto el bit cero que debe quedar como estaba. Verifica el resultado.

Modifica el código para obtener, en la siguiente palabra, que los 16 bits más significativos de numero permanezcan tal cual, y que los 16 bits menos significativos queden a cero, excepto el bit cero que también debe quedar como estaba. No es recomendable utilizar las operaciones lógicas en modo inmediato porque no se puede setear los 16 bits de mayor peso. Recuerde que una constante puede cargarse con la instrucción **li**.

2.3. Operaciones de desplazamiento

Muchas operaciones en el campo de la computación requieren mover lo bits de una cadena de bits hacia la derecha, el bit_{*n*} para a tener el valor del bit_{*n+1*}, o la izquierda, el bit_{*n+1*} para a tener el valor del bit_{*n*}, tambien conocido como desplazamiento (*shift*). En el deplazamiento hacia la derecha (*shift right*) los bits menos significativos se pierden y puede (desplazamiento aritmético) o no (desplazamiento lógico) copiarse el bit de signo en el bit más significativo. De

allí surgen las instrucciones **sra** (*shift right arithmetic*) y **srl** (*shift right logic*). Para el desplazamiento hacia la izquierda (*shift left*) el bit de signo se pierde y se rellenan los bits menos significativos con 0s con la instrucción llamada **sll** (*shift left logic*).

El siguiente código desplaza el valor de contenido en **number** cuatro bits a la derecha, rellenando el hueco generado a su izquierda con el bit del signo. Ejecútalo y analiza los resultados en el segmento de memoria en hexadecimal para entender como opera **sra** y luego observa los mismos en decimal. A continuación experimenta con diferentes valores en **number** negativos y positivos. Identifica con que operación aritmética se relaciona (ayuda $2^4 = 16$). Luego, usando el valor **0xfffff41** en **number** reemplaza la instrucción **sra** con la instrucción **srl** y observa que ocurre con el signo del resultado.

```

        .data
number:  .word 0xfffff41
        .space 4
        .text
main:    lw $s0, number
        sra $s1, $s0, 4
        sw $s1, number+4

```

Modifica el código anterior para desplazar el contenido de **number** 2 bits a la izquierda. Identifica que operación aritmética devuelve el mismo resultado.

Relaciona todo lo ensayado con las divisiones y multiplicaciones de divisores potencia de 2. ¿Puedes generalizar alguna propiedad?

2.4. Problemas propuestos de la sección

1. Diseña un programa ensamblador que defina el vector de enteros de dos elementos 10, 20 en la memoria de datos a partir de la dirección **0x10000000** y almacene su suma a partir de la dirección donde acaba el vector.
2. Diseña un programa ensamblador que divida los enteros 18 y -1215 almacenados a partir de la dirección **0x10000000** por el número 5 y que a partir de la dirección **0x10010000** almacene el cociente de dichas divisiones.
3. Diseña un programa ensamblador que ponga a cero los bits 3, 7, 9 del entero **0xabcd12bd** almacenado en memoria a partir de la dirección **0x10000000**, sin modificar el resto.
4. Diseña un programa ensamblador que cambie el valor de los bits 3, 7, 9 del entero **0xff0f1235** almacenado en memoria a partir de la dirección **0x10000000**, sin modificar el resto.
5. Diseña un programa ensamblador que multiplique el número **0x1237**, almacenado en memoria a partir de la dirección **0x10000000**, por 32 (2^5), sin utilizar las instrucciones de multiplicación ni las pseudoinstrucciones de multiplicación.